

Spark 原理详解

第 14 讲：Shuffle、Partition 与数据倾斜

从“相同 Key 必须相遇”到 Shuffle 性能瓶颈

本讲位置：第 14 / 20 讲。完成本讲后，课程还剩 6 讲：DataFrame & Spark SQL、Catalyst、Spark on YARN、性能优化、综合实战、总复习。

学习目标：学完后，你应该能够看到一个 RDD 操作链，判断哪里会发生 Shuffle；解释 reduceByKey 为什么通常优于 groupByKey 做聚合；理解 Partitioner、Partition 数量和数据倾斜之间的关系；并能在 Spark UI 中用 Shuffle Read/Write 和 Task 时长定位问题。

本讲不是记 API，而是理解一个底层问题：分布式系统里，数据原本分散在不同机器上；当计算要求“相同 Key 必须一起处理”时，系统到底要怎样把它们重新组织起来？

1. 为什么 Spark 需要 Shuffle：相同 Key 原本不在一起

上一讲我们已经知道：map、filter 等窄依赖操作通常可以让一个 Partition 在本地继续往下算；但 reduceByKey、groupByKey、join 等操作会遇到完全不同的问题——同一个 Key 的记录可能散落在很多上游 Partition 中。

例如统计商品销量，P0、P1、P2 都可能出现 product_001。如果不把这些记录汇集到同一个目标 Partition，就无法得到 product_001 的全局总销量。

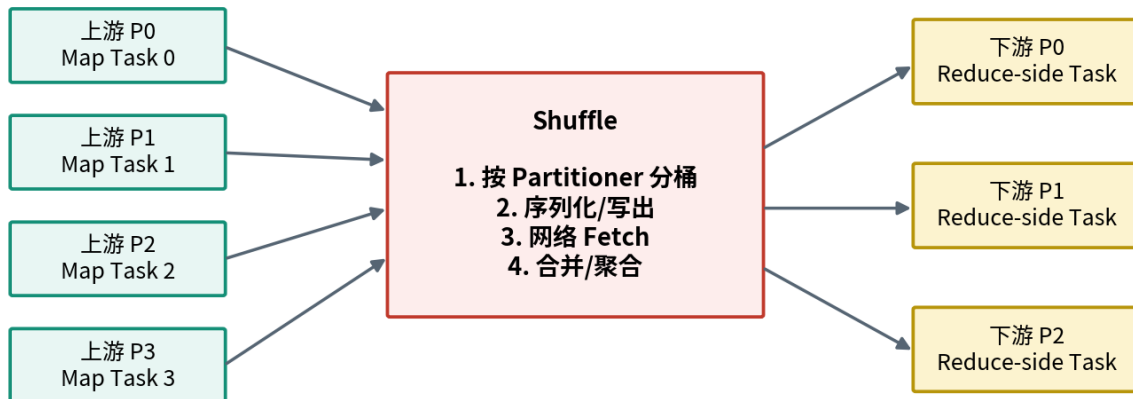
```
P0: (A, 3) (B, 2)
P1: (A, 5) (C, 7)
P2: (B, 4) (A, 1)
```

目标：

```
A -> 3 + 5 + 1
B -> 2 + 4
C -> 7
```

一句话理解：Shuffle 就是为了满足新的数据组织要求，把记录按照目标分区规则重新分发。它不是“排序”本身，也不是“网络传输”本身，而是一整套数据重分布过程。

Spark Shuffle：上游 Partition 按 Key 重新分发到下游 Partition



核心：数据不是“顺着原分区继续算”，而是为了让相同 Key 相遇而重新洗牌。

图1 Spark Shuffle 的总体数据流

2. Shuffle 的完整过程到底发生了什么

为了方便理解，可以把一次典型的 Key-Value Shuffle 拆成五步。不同 Spark 版本的内部文件组织和实现细节会变化，但下面这条逻辑长期稳定。

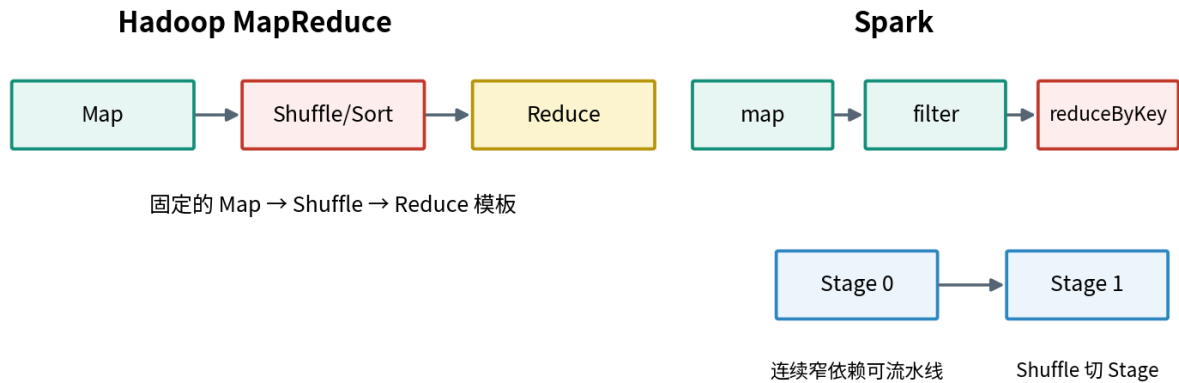
1. 上游 Task 处理自己的 Partition，产生中间 Key-Value。
2. 根据 Partitioner 计算每条记录应该进入哪个下游 Partition。
3. 上游把属于不同目标 Partition 的数据组织成 Shuffle 输出；数据量大时会涉及序列化、内存缓冲和本地磁盘。
4. 下游 Task 从多个上游 Task 获取属于自己的 Shuffle Block，这一步可能发生跨节点网络传输。
5. 下游对取得的数据进行合并、聚合或排序，然后继续执行后续计算。

关键观察：一个下游 Partition 的数据，往往来自很多上游 Partition。因此 Shuffle 会打破“一个分区沿本地流水线一直算下去”的模式，也因此通常形成新的 Stage。

3. Hadoop MapReduce Shuffle 与 Spark Shuffle 有什么关系

如果你已经学懂前面的 Hadoop Shuffle，那么 Spark Shuffle 并不是一个全新的世界。它们的核心思想高度一致：按照 Key 或新的分区规则重新组织数据。

MapReduce Shuffle 与 Spark Shuffle：思想相同，执行模型不同



共同点：相同 Key 的数据要重新分区并跨节点搬运；差异：Spark 把 Shuffle 嵌入更一般的 DAG。

图 2 Hadoop MapReduce Shuffle 与 Spark Shuffle 的关系

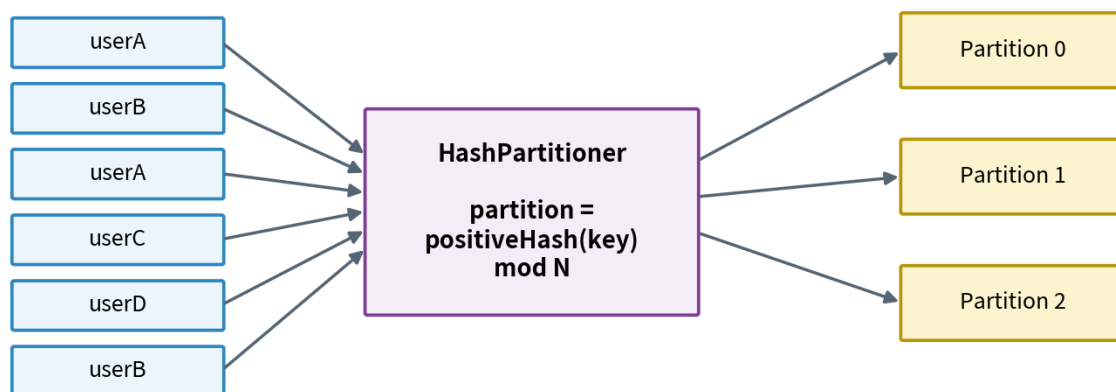
比较项	Hadoop MapReduce	Spark
执行模型	固定 Map → Shuffle/Sort → Reduce	一般 DAG；一次 Job 中可有多个 Shuffle
Stage 概念	通常不以 Spark 的 Stage 形式表达	ShuffleDependency 通常切分 Stage
Map 端聚合	Combiner 可选，且不保证一定执行	reduceByKey 等聚合算子会进行本地合并
中间数据	Map 输出通常落本地磁盘	可能利用内存，但 Shuffle 仍可能大量使用磁盘和网络
核心不变量	同 Key 数据必须到同一 Reducer	同 Key 数据必须到同一目标 Partition

迁移理解：你在 MapReduce 里学过的 Partition、Shuffle、数据倾斜、网络 I/O、磁盘 I/O、局部聚合，在 Spark 中全部仍然存在；只是 Spark 把这些机制放进了更灵活的 DAG。

4. Partitioner：到底谁决定数据去哪儿

Shuffle 之前必须先回答一个非常具体的问题：当前这条记录应该发给哪个目标 Partition？这就是 Partitioner 的职责。

Partitioner 决定“这个 Key 应该去哪个下游 Partition”



正确性底线：同一个 Key 必须稳定地落到同一个目标 Partition。

图3 Partitioner 根据 Key 决定目标 Partition

最常见的思想是 Hash Partition：对 Key 求哈希，再映射到 0 到 N-1 的目标分区。对初学者而言，不需要纠结具体哈希函数实现，重要的是两个性质：

- 同一个 Key 必须稳定地映射到同一个目标 Partition，否则同 Key 无法完整聚合。
- 不同 Key 可以落到同一个 Partition；Partition 并不是“一个 Key 一个分区”。
- 目标 Partition 数量直接影响后续并行 Task 数量、每个 Task 的数据规模和 Shuffle 文件组织。

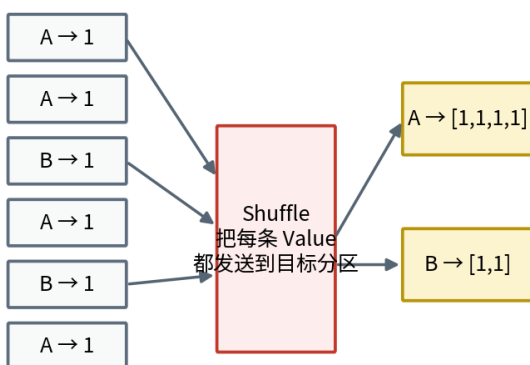
5. reduceByKey 和 groupByKey : 为什么只差一个 API , 代价可能差很多

这是学习 Spark Shuffle 时最重要的一组对比。假设我们的目标只是求每个 Key 的总和：

```
rdd = sc.parallelize([
    ("A", 1), ("A", 1), ("B", 1),
    ("A", 1), ("B", 1), ("A", 1)
], 2)
```

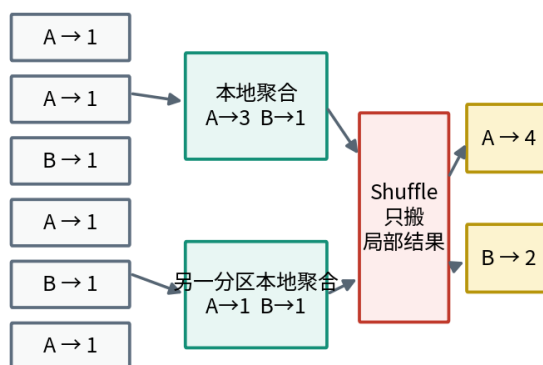
写法一可以是 groupByKey() 后再 sum；写法二可以直接 reduceByKey(lambda a, b: a + b)。两者最终数学结果可能相同，但数据移动方式不同。

groupByKey : 先把所有 Value 搬过去



网络数据量大；单个 Key 的 Value 很多时还可能占用大量内存

reduceByKey : 可先在本地合并



当运算可结合、可交换时，本地合并可显著减少 Shuffle 数据

图 4 groupByKey 与 reduceByKey 的 Shuffle 差异

核心原理：reduceByKey 的函数要求满足结合律与交换律，因此 Spark 可以先在每个上游 Partition 内把同 Key 的 Value 局部合并，再把较小的中间结果发送到网络。groupByKey 则需要保留每个 Value，无法把“完整值集合”提前压缩成一个局部结果。

```
# 只需要“每个商品总销量”时，更自然：
sales.reduceByKey(lambda a, b: a + b)
```

```
# 如果业务真的需要同 Key 的全部原始 Value：
groups = sales.groupByKey()
```

6. groupByKey 不是“错误 API”：关键看你到底需要什么

不能机械地背“永远不要用 groupByKey”。如果需求是得到某个 Key 的全部明细，或者后续逻辑确实需要完整 Value 集合，groupByKey 可能是合理的。问题在于：如果最终只做 sum、max、min、count 之类可增量聚合的计算，却先把所有 Value 全部搬到一起，通常是在制造不必要的 Shuffle 和内存压力。

判断问题：问自己：“我真的需要保存这个 Key 的每一条 Value 吗？还是只需要一个可以逐步合并的状态？”如果只需要状态，就优先考虑 reduceByKey、aggregateByKey、combineByKey 等聚合思路。

7. 一次 Shuffle 为什么昂贵

官方 Spark RDD 编程指南把 Shuffle 描述为一种 expensive operation。它贵，不是因为某个函数的数学运算特别复杂，而是它同时触碰分布式系统里最昂贵的几种资源。

- 网络 I/O：下游 Partition 的数据可能来自其他 Executor 所在节点。
- 序列化与反序列化：对象必须转换成可传输/可存储的字节，再恢复。
- 磁盘 I/O：Shuffle 数据无法完全留在内存时，需要 Spill 或本地文件。
- 内存压力：排序、聚合、拉取和合并都可能需要缓冲区。
- 调度边界：Shuffle 往往意味着上游 Stage 与下游 Stage 之间存在依赖等待。

所以在性能分析里，一个非常实用的思路是：先数 Shuffle，再看每个 Shuffle 到底移动了多少数据。

不要误解：“避免 Shuffle”不是绝对目标。很多真正有价值的操作（聚合、Join、全局重分区）本来就需要 Shuffle。优化的目标是：避免不必要的 Shuffle、减少 Shuffle 数据量，并让 Shuffle 后的 Partition 尽量均衡。

8. Partition 数量：太少和太多都会出问题

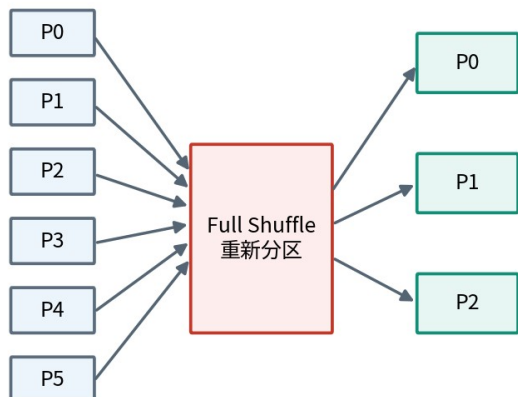
Shuffle 后有 N 个目标 Partition，通常意味着该 Stage 会有大约 N 个 Task。于是 Partition 数量直接成为“并行度”和“单 Task 数据规模”的控制旋钮。

Partition 太少	Partition 太多
每个 Task 数据过大	Task 数量和调度开销上升
并行度不足，CPU 空闲	产生大量很小的 Shuffle Block/输出文件
单 Task 更容易 Spill / OOM	启动、序列化、调度成本相对变高
少数慢 Task 更容易拖尾	下游小文件问题更明显

先建立直觉：Partition 不是越多越好，也不是越少越好。合理值依赖总数据量、单条记录大小、集群 CPU 核数、算子成本和下游输出要求。第 18 讲性能优化会系统讲如何估算与观察。

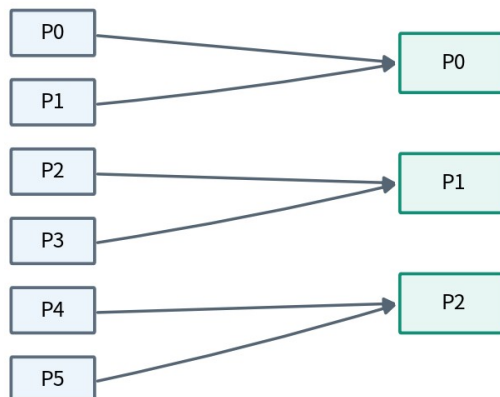
9. repartition() 与 coalesce()：都是改分区数，为什么不一样

repartition(3)：重新洗牌并均衡分布



更均衡，但需要 Shuffle 成本

coalesce(3)：常用于减少分区



默认可避免完整 Shuffle，但可能不够均衡

图5 repartition 与 coalesce 的典型区别

repartition(n) 的目标是重新打散并形成新的 n 个 Partition，因此会触发 Shuffle，代价较高，但通常可以获得更均衡的数据分布。coalesce(n) 常用于减少 Partition；默认情况下尽量避免完整 Shuffle，因此更便宜，但有可能让新的 Partition 大小不均。

```
# 例如从 100 个分区重新均衡成 20 个  
r2 = rdd.repartition(20)
```

```
# 只是希望把很多小分区合并成较少分区  
r3 = rdd.coalesce(20)
```

10. 数据倾斜：为什么 99 个 Task 都完了，最后 1 个还跑很久

假设我们有 100 个目标 Partition，其中 99 个只有几百 MB，而某个热点 Key 导致 Partition 37 有 40 GB。Spark 的一个 Stage 必须等所有 Task 完成，所以整个 Stage 的完成时间接近那个最慢 Task 的完成时间。

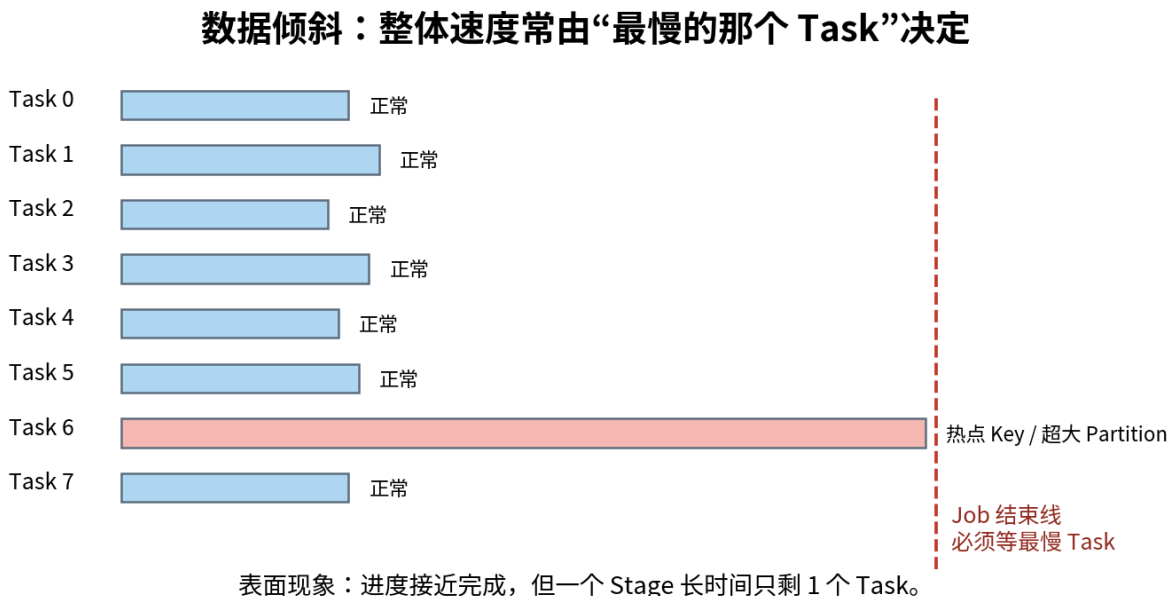


图6 数据倾斜导致长尾 Task

常见热点 Key 包括：unknown、NULL、默认城市、超级热门商品、大客户、机器人账号、某个异常日期等。它们的问题不是 Key 本身“特殊”，而是这个 Key 对应的数据量远超其他 Key。

最危险的误区：数据倾斜时单纯增加 Partition 数量不一定有效。只要同一个热点 Key 仍必须完整落在一个目标 Partition，它依然可能把那个 Partition 撑得很大。

11. 数据倾斜怎么处理：从最便宜的方法开始

- 先确认倾斜真的存在：在 Spark UI 看 Task Duration、Input Size、Shuffle Read/Write 是否出现极端离群值。
- 能过滤就尽量提前过滤：把无用数据在 Shuffle 之前删掉。
- 如果只是聚合，优先使用可本地合并的算子，如 `reduceByKey`，而不是把全部 Value 直接 `groupByKey`。
- 检查 Key 设计：例如把大量空值统一映射成 NULL，可能人为制造热点。
- 对可拆分的热点 Key 使用 Salting（加盐）+ 两阶段聚合，把一个超级 Key 拆成多个临时 Key。
- 如果是 Join 导致倾斜，后续 DataFrame/Spark SQL 课程会学习 Broadcast Join、AQE 等更现代的处理方法。

```
# 加盐的思想示意:  
# 原来: ("hot", value) 全部去同一个分区  
# 第1阶段: ("hot#0", value), ("hot#1", value), ...  
# 局部聚合后再去掉盐值  
# 第2阶段: ("hot", partial_sum) 再聚合一次
```

正确性提醒：加盐只能用于“结果可分解再合并”的问题。SUM/COUNT 很自然；AVG 要携带 (sum, count) 再合并，不能直接平均多个局部平均值。

12. Spark UI 中怎样判断 Shuffle 问题

真正排查性能时，不要先猜配置。先从 Web UI 的 Stage 和 Task 指标寻找证据。你暂时可以重点看下面几个字段：

观察项	说明	异常意味着什么
Shuffle Write	上游 Stage 写出的 Shuffle 数据量	过大：中间结果太多、局部聚合不足
Shuffle Read	下游 Stage 读取的 Shuffle 数据量	过大：网络/磁盘压力增大
Task Duration	每个 Task 的运行时间	少数 Task 极慢：可能倾斜或节点问题
Records	Task 处理的记录数	极端不均：很可能 Partition 不均
Spill	内存不足后落盘的数据	大量 Spill：内存压力或单分区过大

排查顺序：先看哪个 Stage 慢 → 再看是否有 Shuffle → 看 Shuffle Read/Write → 看 Task 是否长尾 → 看记录数/数据量是否倾斜 → 最后再决定要改算子、Key、Partition 数还是资源配置。

13. 一个可以直接运行的 PySpark 实验

下面这个实验不是为了跑出“性能排行榜”，而是让你亲眼看到分区、reduceByKey 和 groupByKey 的行为差异。

```
# 启动 pyspark 后执行
data = [
    ("A", 1), ("A", 1), ("B", 1), ("A", 1),
    ("C", 1), ("A", 1), ("B", 1), ("A", 1)
]

rdd = sc.parallelize(data, 4)
print("输入分区数:", rdd.getNumPartitions())

# 方案 1: 直接聚合
r1 = rdd.reduceByKey(lambda a, b: a + b, numPartitions=2)
print("reduceByKey 分区数:", r1.getNumPartitions())
print(sorted(r1.collect()))

# 方案 2: 先分组
r2 = rdd.groupByKey(numPartitions=2)
r2_sum = r2.mapValues(lambda xs: sum(xs))
print(sorted(r2_sum.collect()))
```

两种写法的结果都应该得到 A=5、B=2、C=1。但真正的大数据场景下，你应该关注：网络上到底搬了多少 Value、单个 Key 的值集合有多大、是否存在局部聚合机会。

14. 再做一个数据倾斜实验

```
# 构造一个明显热点 Key
```

```
skew = [("hot", 1)] * 100000 + [("k" + str(i), 1) for i in range(10000)]  
rdd = sc.parallelize(skew, 8)  
  
result = rdd.reduceByKey(lambda a, b: a + b, numPartitions=8)  
print(result.take(10))
```

如果在本地模式数据量不够大，你未必能看到明显长尾；这个实验的目的主要是建立结构直觉。真正集群中要结合 Spark UI 查看各 Task 的 Shuffle Read、Records 和 Duration。

15. 常：把 Shuffle 学成几个错误口号

误区 1：Shuffle 就是排序。 不对。Shuffle 的核心是重新分区和数据搬运；某些执行路径会伴随排序，但“重新组织数据”才是本质。

误区 2：Spark 是内存计算，所以 Shuffle 不落盘。 不对。内存不足或实现需要时，Shuffle 会大量使用本地磁盘；Spark 也需要网络传输和序列化。

误区 3：reduceByKey 永远比 groupByKey 好。 不对。若业务真的需要一个 Key 的完整 Value 集合，groupByKey 有合理场景。只是做聚合时 reduceByKey 通常更合适。

误区 4：Partition 越多越快。 不对。过多 Partition 会增加调度、序列化和小文件成本。

误区 5：数据倾斜加几个 Executor 就能解决。 不一定。一个巨大 Partition 通常仍只能由一个 Task 处理，资源增加无法自动拆开热点 Key。

误区 6：看到 Shuffle 就说明程序写错了。 不对。聚合、Join、全局重分区等任务本来就需要 Shuffle。重要的是判断 Shuffle 是否必要、数据量是否合理、分区是否均衡。

16. 把本讲压缩成一张思维链

```
业务要求“相同 Key 必须一起处理”
↓
需要重新组织数据
↓
Partitioner 决定目标 Partition
↓
Shuffle: 写出 + 网络 Fetch + 合并/聚合
↓
Wide Dependency / Stage 边界
↓
性能成本：网络 + 磁盘 + 序列化 + 内存
↓
如果 Partition 不均 → 数据倾斜 → 长尾 Task
↓
优化：减少数据 → 本地聚合 → 合理分区 → 处理热点 Key
```

本讲最低闭环：看到 reduceByKey/groupByKey/join/repartition 时，先问三个问题：①是否会 Shuffle？②Shuffle 多少数据？③ Shuffle 后各 Partition 是否均衡？这三个问题已经能解释大量 Spark 性能现象。

17. 练习题

- 1) 有 20 亿条订单，只求每个商品总销售额。你更倾向 groupByKey 还是 reduceByKey？为什么？
- 2) 为什么同一个 Key 必须进入同一个目标 Partition，但一个 Partition 可以包含多个不同 Key？

- 3) 一个 Stage 有 200 个 Task，其中 199 个都在 30 秒内完成，最后 1 个运行 12 分钟。你首先怀疑什么？应该在 UI 看哪些指标？
- 4) 从 1000 个小 Partition 减少到 50 个，仅为降低小任务/小文件数量，你会先考虑 coalesce 还是 repartition？什么情况下反而应该 repartition？
- 5) 求平均值时对热点 Key 做“加盐 + 两阶段聚合”，中间状态为什么应该是 (sum, count)，而不是 local_avg？

18. 参考答案

12. 1) 通常 reduceByKey。因为求和满足结合律和交换律，可先在上游 Partition 内做局部合并，减少 Shuffle Value 数量。
13. 2) 聚合正确性要求同一个 Key 的所有记录最终相遇；但一个 Task/Partition 可以依次处理很多不同 Key，因此无需“一 Key 一 Partition”。
14. 3) 优先怀疑数据倾斜或单节点异常。查看该 Task 的 Records、Input Size、Shuffle Read/Write、Spill、Duration，并与其他 Task 对比。
15. 4) 减少分区且不要求重新均衡时先考虑 coalesce；如果旧分区本身非常不均、希望重新打散，repartition 更合适。
16. 5) 平均值不是可直接“平均平均值”的可结合状态。必须保留总和与样本数，最终计算 $\text{total_sum} / \text{total_count}$ 。

19. 本讲总结：真正理解 Shuffle，要抓住四个词

重分区：数据为了满足新的 Key 组织规则，从原 Partition 重新映射到目标 Partition。

数据移动：一旦目标数据不在本机，就需要跨 Executor/节点 Fetch；这带来网络、序列化和磁盘成本。

均衡：Partition 数量只是并行度的一半，数据是否均匀决定 Task 是否真正均衡。

局部聚合：能在 Shuffle 前把很多 Value 压缩成更小状态，是 reduceByKey 等算子的重要优势。

你现在应该能回答：为什么 reduceByKey 比 groupByKey 更适合做求和？为什么 repartition 会触发 Shuffle？为什么数据倾斜不是简单“多加几台机器”就一定解决？为什么 Spark 的内存计算仍然离不开磁盘和网络？

20. 下一告：DataFrame 与 Spark SQL

RDD 帮我们理解了 Spark 最底层的分区、依赖、Shuffle 和容错。但现实工程中，大量结构化数据分析不会直接手写 RDD，而会使用 DataFrame 与 Spark SQL。下一讲会回答：

- DataFrame 为什么比 RDD 更像“分布式表”？
- Schema、Row、Column 到底是什么？
- 同样的过滤、聚合、Join，DataFrame 为什么能让 Spark 自动优化？
- PySpark DataFrame 与 SQL 为什么可以互相切换？
- RDD 的 Partition、Shuffle、Stage 知识在 DataFrame 中是否仍然有效？

课程进度：第 14 讲完成后，还剩 6 讲（15~20 讲）。第 20 讲完成 MapReduce + Spark 总闭环，然后再进入 Flink。

参考资料

本讲核心机制依据 Apache Spark 官方 RDD Programming Guide 与当前 PySpark RDD API 文档整理。官方文档将 Shuffle 描述为跨 Partition 重分布数据的高成本操作，并指出其涉及磁盘 I/O、序列化和网络 I/O；reduceByKey 支持在发送前进行本地合并；repartition 会重新洗牌数据，而 coalesce 可用于更经济地减少分区。

- Apache Spark RDD Programming Guide（当前官方文档）
- PySpark RDD.reduceByKey API
- PySpark RDD.groupByKey API
- PySpark RDD.repartition / coalesce API